



TEL AVIV אוניברסיטת  
UNIVERSITY תל אביב

## מבני נתונים –

## תרגיל בית מעשי 1

מגישים (שם, ת"ז, שם משתמש):

יעקב חזון – 208234161 – kobiehazon

יצחק הראל – 315177212 - itzchakharel

תאריך הגשה: 30.12.2018

# מסמך התייעוד הנלווה לתרגיל מעשי 1 – מימוש עץ WAVL

## תיעוד

כפי שהתבקשנו, מימשנו את הדרוש באמצעות שתי מחלקות ששמן:

- **WAVLTree** שמתארת את עץ ה-WAVL.
- **WAVLNode** שמתארת כל צומת בעץ וממנה מורכב העץ.

כמו כן, כפי שהופיע בקובץ השלד, המחלקה WAVLNode נכתבה בתור מחלקה פנימית של המחלקה WAVLTree.

## הסבר על מחלקת WAVLNode:

כאמור, מחלקה זו מתארת את צמתי העץ ומכאן שכוללת את השדות הבאים:

- **key** - מטיפוס *int*. תפקידו לשמור את מפתח הצומת.
- **value** - מטיפוס *String*. תפקידו לשמור את ערך הצומת.
- **Rank** - מטיפוס *int*. תפקידו לשמור את הדרגת הצומת.
- **left** - מטיפוס *WAVLNode*. תפקידו לשמור מצביע לבן השמאלי של הצומת.
- **right** - מטיפוס *WAVLNode*. תפקידו לשמור מצביע לבן הימני של הצומת.
- **parent** - מטיפוס *WAVLNode*. תפקידו לשמור מצביע לאבא של הצומת.
- **size** - מטיפוס *int*. תפקידו לשמור את גודל תת העץ של צומת זה על מנת שנוכל להביאו בסיבוכיות  $O(1)$ .

יתרה מכך, במחלקה מימשנו את המתודות הבאות:

○ getKey() -

**מה היא עושה-**

מתודה זו מחזירה את המפתח של הצומת. באופן טבעי, המתודה תחזיר 1- אם הצומת היא חיצונית שכן צומת חיצונית היא גם צומת לפי הקוד שכתבנו (ממומשת עם WAVLNode).

**כיצד היא פועלת-**

מחזירה את שדה ה-key של הצומת.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן נעשית רק החזרה של שדה המופע.

## ○ - getValue()

**מה היא עושה-**

מחזירה את שדה ה-value של הצומת. באופן טבעי המתודה תחזיר null עבור צומת חיצוני. שכן צומת חיצוני הוא צומת לכל דבר לפי הקוד שכתבנו כאשר ה-value שלו הוא null.

**כיצד היא פועלת-**

מחזירה את השדה value של הצומת.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן נעשית רק החזרה של שדה המופע.

## ○ - getLeft()

**מה היא עושה-**

מחזירה את הצומת השמאלי של הצומת או null אם הוא לא קיים.

**כיצד היא פועלת-**

מחזירה את השדה left של הצומת.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן נעשית רק החזרה של שדה המופע.

## ○ - getRight()

**מה היא עושה-**

מחזירה את הצומת הימני של הצומת או null אם הוא לא קיים.

**כיצד היא פועלת-**

מחזירה את השדה right של הצומת.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן נעשית רק החזרה של שדה המופע.

## ○ isInnerNode() -

**מה היא עושה-**

מחזירה אם הצומת פנימית או חיצונית. הפונקציה מחזירה ערך בוליאני כאשר אם הצומת פנימית יוחזר true ואחרת יוחזר false.

**כיצד היא פועלת-**

מחזירה האם השדה rank שונה מ-1 - זאת מכיוון שה-rank של צומת שווה ל-1 - רק אם היא חיצונית.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן רק נעשית השוואה על פי שדה מופע.

## ○ getSubtreeSize() -

**מה היא עושה-**

מחזירה את גודל תת העץ המתחיל מהצומת הנוכחית (כלומר נסתכל על הצומת הנוכחית כשורש ונחזיר את גודל העץ שלה).

**כיצד היא פועלת-**

שמרנו שדה מופע המציין את גודל תת העץ אשר אותו אנו מעדכנים בעת הכנסה ומחיקה של צומת ולכן כל שעלינו לעשות הוא להחזיר שדה זה.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן רק מתבצעת החזרה של שדה מופע.

\* חשוב לציין כי במחלקה זו יש עוד מתודות שיעזרו לביצוע מתודות במחלקה WAVLtree שעליהן נסביר בהמשך.

## הסבר על מחלקת WAVLTree:

מחלקה זו מתארת את העץ ה-WAVL ועושה שימוש במופעים של WAVLNode וכן במתודות המצויות במחלקה זו.

במחלקה זו מצויים השדות הבאים:

- **root** - מטיפוס WAVLNode תפקידו הוא לשמור מצביע לשורש העץ.
- **size** - מטיפוס int. תפקידו הוא לשמור את גודל העץ (מספר הצמתים הפנימיים שיש בעץ).
- **min** - מטיפוס WAVLNode שישמור מצביע לצומת בעץ שהמפתח שלו מינימלי.
- **max** - מטיפוס WAVLNode שישמור מצביע לצומת בעץ שהמפתח שלו מקסימלי.
- **enum NodeType** - תפקידה לעזור לנו לדעת "מה סוג הצומת" ובהתאם יש לה ערכים את סוג הצומת (עלה, אונארי או מלא וכל צד) במתודה בתוך המחלקה ששמה getType() נבדוק מה ה"סוג" של כל צומת וניעזר ב-enum זה כדי לדעת מה עלינו לעשות במחיקה או איזון.
- **enum InsertRebalanceCase** - תפקידה לעזור לנו לדעת מהי פעולת האיזון הנדרשת לביצוע בעת הכנסה. בהתאם, יש לה ערכים התואמים את המקרים המופיעים במצגת הקורס. במתודה checkInsertRebCase נבדוק מה סוג ה-rebalance אשר עלינו לבצע וניעזר במחלקת enum זאת.
- **enum DeleteRebalanceCase** - תפקידה לעזור לנו לדעת מהי פעולת האיזון הנדרשת לביצוע בעת מחיקה. בהתאם, יש לה ערכים התואמים את המקרים המופיעים במצגת הקורס. במתודה checkDeleteRebCase נבדוק מה סוג ה-rebalance אשר עלינו לבצע וניעזר במחלקת enum זאת.
- **RIGHT** - שדה סטטי וסופי מטיפוס בוליאני שמאותחל לערך אמת. תפקידו הוא לעזור לנו לדעת על איזה צד של העץ עלינו לבצע מספר פונקציות (שכן יש סימטריה רבה בפעולות האיזון ובהתחייסות במתודות).
- **LEFT** - שדה סטטי וסופי מטיפוס בוליאני שמאותחל לערך ההפוך מ-RIGHT. תפקידו לעזור לנו לדעת על איזה צד של העץ עלינו לבצע מספר פונקציות (שכן יש סימטריה רבה בפעולות האיזון ובהתחייסות במתודות).
- **INSERT** - שדה סטטי וסופי מטיפוס בוליאני שמאותחל לערך אמת. תפקידו לעזור לנו לדעת בפעולות האיזון האם קראנו להם מתוך הכנסה או הוצאה ולכן לבצע הגדלת והקטנת דרגות בהתאם (שכן יש סימטריה רבה בפעולות האיזון ובהתחייסות במתודות).
- **DELETE** - שדה סטטי וסופי בוליאני שמאותחל לערך ההפוך מ-INSERT. תפקידו לעזור לנו לדעת בפעולות האיזון האם קראנו להם מתוך הכנסה או הוצאה ולכן לבצע הגדלת והקטנת דרגות בהתאם (שכן יש סימטריה רבה בפעולות האיזון ובהתחייסות במתודות).

יתרה מכך, במחלקה זו מימשנו את המתודות הבאות:

## ○ empty() -

**מה היא עושה-**

מחזירה true אם העץ ריק אחרת יוחזר הערך false.

**כיצד היא פועלת-**

מחזירה את הערך true אם השדה root שווה לnull.

**סיבוכיות זמן הריצה שלה-**

$O(1)$  שכן מתבצעת רק פעולה אחת לכל קלט.

## ○ search() -

**מה היא עושה-**

מחפשת איבר בעץ שהמפתח שלו שווה למספר שלם k. אם האיבר נמצא יוחזר ערך value של אחרת יוחזר null.

**כיצד היא פועלת-**

בודקת אם העץ ריק בעזרת קריאה למתודה empty() שתוארה לעיל ואם העץ אכן ריק יוחזר הערך null. אחרת מזמנים את פונקציית העזר searchSubtree(k) על השדה root של העץ. הפונקציה searchSubtree(k) מחזירה מצביע לאיבר שהמפתח שלו שווה ל-k אם היא מצאה אחד כזה ואחרת מחזירה null, לפיכך, אם searchSubtree(k) מצאה איבר בעץ שהמפתח שלו k והחזירה אותו נחזיר את ה-value של אותו האיבר. אחרת אם searchSubtree(k) החזירה null נחזיר null.

הסבר על פונקציית העזר searchSubtree(k) בה השתמשנו- הפונקציה הזו עוזרת למספר פונקציות. היא מקבלת ערך k וחזירה מצביע לאיבר בתת העץ של הnode שעליו הפונקציה פועלת שהמפתח שלו שווה ל-k. הפונקציה עובדת בדומה לאופן בו מתבצע חיפוש בינארי. כל עוד הצומת שקיבלנו היא פנימית וה-key שקיבלנו שונה מהמפתח של הצומת שקיבלנו נבצע את התהליך הבא: אם k גדול מהמפתח של הצומת שקיבלנו נחזור על התהליך עם הבן הימני של הצומת שקיבלנו ואחרת עם הבן השמאלי. אם מצאנו בתהליך את הצומת הרצוי נחזיר אותו ואחרת יוחזר הערך null.

**סיבוכיות זמן הריצה שלה-**

$O(\log(n))$  כאשר n הוא מספר הצמתים הכולל של העץ שכן במקרה הגרוע נעבור על מספר צמתים ששווה לגובה הכולל של העץ והגובה של העץ שווה ל- $\log(n)$ .

**קריאה לפונקציות עזר-** יש קריאה לפונקציה searchSubtree ולפונקציה empty שעל שתיהן ניתן הסבר לעיל.

## ○ insert() -

### מה היא עושה-

מכניסה איבר בעל ערך  $i$  ומפתח  $k$  לעץ אם הוא לא קיים כבר בעץ. הפונקציה תחזיר את מספר פעולות האיזון שנדרשו לביצוע ההכנסה. אם קיים איבר בעץ ששווה לאיבר אותו אנו נדרשים להכניס נחזיר 1-.

### כיצד היא פועלת-

נבצע קריאה לפונקציית העזר  $\text{insertBinarySearchTree}(k, i)$  שתכניס את הצומת לעץ במידה וצומת עם ערכים זהים קיים כבר בעץ היא תחזיר null ואחרת היא תכניס את הצומת לעץ. הפונקציה  $\text{insertBinarySearchTree}(k, i)$  תבצע קריאה למספר פונקציות עזר. ראשית תבצע קריאה לפונקציה  $\text{empty}$  שלפי התוצאה שלה נדע אם מדובר בעץ ריק. אם אכן הוא ריק, תבצע קריאה לפונקציה  $\text{setRoot}$  שתקבל את הצומת אותו אנו רוצים להכניס ותקבע שהשורש של העץ יהיה הצומת שאנו רוצים להכניס. בנוסף תבצע קריאה לפונקציה  $\text{searchPosition}(\text{root}, k)$  שתחפש מה מקום ההכנסה הדרוש. יתר על כן תבצע קריאה לפונקציית העזר  $\text{setSon}$  שתוסיף את הצומת הדרוש לעץ. הפונקציה תעדכן כמובן את השדות  $\text{min}$  ו- $\text{max}$  בהתאם לצומת שהוכנס. במידה שאכן מתבצעת הכנסה, תבצע קריאה לפונקציה  $\text{RebalanceInsert}(\text{currPos.parent})$ . פונקציה זו תבצע קריאה לפונקציה  $\text{checkInsertRebCase}(\text{currNode})$  שתבדוק באיזה מקרה של  $\text{rebalance}$  אנחנו נמצאים ולפי המקרה יתבצע האיזון הנדרש. אצוין כי לאחר שאיזון מסוים התבצע יכול להיות שנידרש לביצוע איזון נוסף ולכן הפונקציה  $\text{RebalanceInsert}$  הינה רקורסיבית, כך שלאחר שהתבצע איזון מסוים נבדוק אם נידרש לבצע איזון חדש עד אשר נגיע למקרה בו לא נידרש לאזן את העץ כלל. בפעולות האיזון יש קריאה למספר פונקציות עזר אשר תפקידן לבצע "סיבובים" בעץ. פונקציות אלו הן:  $\text{RotateLeft}$ ,  $\text{RotateRight}$ ,  $\text{DoubleRotateLeft}$  ו- $\text{DoubleRotateRight}$ . נזכור לסכום את פעולות האיזון שאנו מבצעים ובנוסף נזכור לשנות את השדה  $\text{size}$  כך שאם הוספנו איבר לעץ ערכו יקטן ב-1 וכן נעדכן את ערך גודל תת העץ של כל איבר מהצומת שהוכנסה ועד לשורש.

### סיבוכיות זמן הריצה-

$O(\log(n))$ . סיבוכיות זמן הריצה של הפונקציה  $\text{insert}$  במקרה הגרוע היא  $O(\log(n))$  כאשר  $n$  מייצג את מספר הצמתים שיש בעץ. זאת משום שבכדי למצוא את מקום ההכנסה עלינו לעבור במקרה הגרוע על מספר צמתים אשר שווה לגובה העץ. בביצוע הפונקציה שמאזנת את העץ נעשה במקרה הגרוע שתי פעולות סיבוב (שכן פעולת הסיבוב היא "טרמינלית" ולאחריה העץ יהיה מאוזן) וכן נבצע במקרה הגרוע מספר פעולות  $\text{promote}$  ששווה לגובה העץ. כמו כן, נבחין כי כל פעולת איזון (כפי שמופיע במצגת) סיבוב/שינוי דרגות כוללת מספר סופי של פעולות.

לכן הסיבוכיות של איזון העץ היא  $O(\log(n))$  במקרה הגרוע.

### קריאה לפונקציות עזר-

אנו השתמשנו במספר רב של פונקציות עזר משום שרצינו שהקוד יהיה ברור וקריא. פונקציות העזר בהן השתמשנו מפורטות תחת הסעיף "כיצד היא פועלת".

## ○ delete()-

### מה היא עושה-

מוחקת איבר בעל מפתח  $k$  מהעץ אם הוא קיים. אחרת יוחזר הערך 1-. במידה ואכן התבצעה מחיקה הפונקציה תחזיר את מספר פעולות האיזון שנדרשו בכדי לבצע את פעולת המחיקה.

### כיצד היא פועלת-

מתבצעת קריאה לפונקציית העזר `empty()` שלפי הערך שיוחזר ממנה נדע אם העץ ריק. אם העץ ריק אין צמתים למחוק ממנו ונחזיר 1-. אחרת נקרא לפונקציית העזר `searchSubtree(k)` שתקבל את השדה של שורש העץ ותחזיר את הצומת שהמפתח שלו הוא  $k$  ותפעל כפי שכבר תיארתי בסעיף "כיצד היא פועלת" של הפונקציה `insert`. אם הצומת בעל המפתח הדרוש לא קיים בעץ (נדע זאת כאשר הפונקציה `searchSubtree(k)` תחזיר null) נחזיר 1-. במקרה שנמצא הצומת שעלינו למחוק נעדכן את השדות `min` ו-`max`. בעזרת פונקציות העזר `predecessor(deletedNode)` ו-`successor(deletedNode)` נוכל לעדכן שדות אלו שכן ככה נוכל למצוא את המקסימום והמינימום החדשים (אם מחקנו את המקסימום נקרא לראשון ואם מחקנו את המינימום נקרא לשני). כעת, נצטרך למחוק את הצומת שנמצאה בעזרת הפונקציה `DeleteBinarySearchTree(deletedNode)` ולאזן את העץ בעזרת הפונקציה `RebalanceDelete`. פונקציה זו תשתמש בפונקציית העזר `checkDeleteRebCase` שבודקת מה האיזון הדרוש ופעולת האיזון של `delete` מאזנת בהתאם. האיזון נערך באופן רקורסיבי עד אשר מגיעים למצב בו אין עוד צורך באיזונים נוספים. בפעולות האיזון יש קריאה למספר פונקציות עזר אשר תפקידן לבצע "סיבובים" בעץ. פונקציות אלו הן: `RotateLeft`, `RotateRight`, `DoubleRotateLeft` ו-`DoubleRotateRight`. נזכור לסכום את פעולות האיזון שאנו מבצעים ובנוסף נזכור לשנות את השדה `size` כך שאם מחקנו איבר מהעץ ערכו יקטן ב-1 וכן נעדכן את שדה גודל תת העץ בכל צומת מאב הצומת המחוקה ועד לשורש בכך שנקטינו ב-1.

### סיבוכיות זמן הריצה-

$O(\log(n))$ . על מנת למצוא את הצומת שעלינו למחוק יש לרדת במקרה הגרוע מספר צמתים ששווה לגובה העץ ולכן סיבוכיות זמן הריצה לעשות זאת היא  $O(\log(n))$ . בנוסף בביצוע המחיקה אם לצומת שאנו רוצים למחוק יש שני ילדים נידרש למצוא את הצומת העוקב לו בסיבוכיות  $O(\log(n))$  (במקרה הגרוע נרד את כל גובה העץ). במקרים האחרים של המחיקה (לצומת יש ילד אחד או שהצומת עלה) מספר פעולות המחיקה יהיה קבוע. בביצוע האיזון כל איזון מכיל מספר פעולות קבוע ונבצע איזון במקרה הגרוע על מספר צמתים אשר שווה לגובה העת ולכן סיבוכיות האיזון במקרה הגרוע תהיה  $O(\log(n))$ .

משום כך הסיבוכיות הכוללת תהיה  $O(\log(n))$  במקרה הגרוע.

### קריאה לפונקציות עזר-

אנו השתמשנו במספר רב של פונקציות עזר משום שרצינו שהקוד יהיה ברור וקריא. פונקציות העזר בהן השתמשנו מפורטות תחת הסעיף "כיצד היא פועלת".

## ○ min() -

**מה היא עושה-**

מחזירה את הvalue של הצומת בעץ בעל המפתח המינימלי או null במקרה בו העץ ריק.

**כיצד היא פועלת-**

מזמנת את הפונקציה empty() שבודקת אם העץ ריק. אם העץ אכן ריק יוחזר null ואם העץ לא ריק יוחזר הvalue של הצומת בשדה min.

**סיבוכיות זמן הריצה-**

$O(1)$  שכן מתבצע מספר קבוע של פעולות בפונקציה.

קריאה לפונקציות עזר- כפי שציינתי, מתבצעת קריאה לפונקציה empty() שבודקת אם העץ ריק.

## ○ max() -

**מה היא עושה-**

מחזירה את הvalue של הצומת בעץ בעל המפתח המקסימלי או null במקרה בו העץ ריק.

**כיצד היא פועלת-**

מזמנת את הפונקציה empty() שבודקת אם העץ ריק. אם העץ אכן ריק יוחזר null ואם העץ לא ריק יוחזר הvalue של הצומת בשדה max.

**סיבוכיות זמן הריצה-**

$O(1)$  שכן מתבצע מספר קבוע של פעולות בפונקציה.

**קריאה לפונקציות עזר-**

כפי שציינו, מתבצעת קריאה לפונקציה empty() שבודקת אם העץ ריק.

## ○ keysToArray() -

**מה היא עושה-**

מחזירה מערך ממוין שמכיל את כל המפתחות של הצמתים הקיימים בעץ או מערך ריק אם העץ ריק.

**כיצד היא פועלת-**

ראשית, הפונקציה מבצעת בדיקה אם השדה size שווה ל-0 כלומר אם העץ ריק. אם העץ אכן ריק יוחזר מערך ריק. אחרת, נבנה מערך בגודל של השדה size ומתבצעת קריאה לפונקציה subtreeKeysToArray(arr, 0) שמקבלת את השדה root, את המערך שבנינו ו-0. ובאמצעות סריקת in-order של העץ מאחסנת את המפתחות של העץ למערך. הערך 0 שהפונקציה קיבלה מסמן את האינדקס הבא במערך בו צריך להכניס ערך של מפתח של צומת בעץ. כמובן שערך זה גדל במהלך ביצוע הפונקציה subtreeKeysToArray. הפונקציה subtreeKeysToArray היא רקורסיבית כך שעל כל צומת פנימי היא מבצעת סריקה מכניסה את המפתח שלו למערך ואם אחד הבנים שלו או שניהם הם צמתים פנימיים בעץ הפונקציה תזומן גם עליהם עם קבלת index מתאים. לבסוף הפונקציה keysToArray תחזיר את המערך שקיבלה מפונקצית העזר subtreeKeysToArray. נציין שהמערך שיוחזר יהיה ממוין כי עברנו על צמתי העץ בסריקת in-order.

**סיבוכיות זמן הריצה-**

במקרה הגרוע(העץ לא ריק) נעבור על כל הצמתים שלו באמצעות סריקת in-order ולכן הסיבוכיות תהיה  $O(n)$  כאשר n הוא מספר הצמתים בעץ.

**קריאה למתודות עזר-**

קראנו למתודת העזר subtreeKeysToArray כפי שציינו.

## ○ infoToArray() -

**מה היא עושה-**

מחזירה מערך של מחרוזות של כל ערכי ה-value של כל הצמתים בעץ. המחרוזות שיהיו במערך המוחזר יהיו ממוינות לפי סדר המפתחות. אם העץ ריק יוחזר מערך ריק.

**כיצד היא פועלת-**

ראשית, הפונקציה מבצעת בדיקה אם השדה size שווה ל-0 כלומר אם העץ ריק. אם העץ אכן ריק יוחזר מערך ריק. אחרת, נבנה מערך בגודל של השדה size ומתבצעת קריאה לפונקציה subtreeValuesToArray(arr, 0) עבור השורש עם המערך שבנינו ו-0. ובאמצעות סריקת in-order של העץ מאחסנת את ערכי ה-value של העץ למערך. הערך 0 שהפונקציה קיבלה מסמן את האינדקס הבא במערך בו צריך להכניס ערך ה-value של צומת בעץ. כמובן שערך זה גדל במהלך ביצוע הפונקציה subtreeValuesToArray. הפונקציה subtreeValuesToArray היא רקורסיבית כך שעל כל צומת פנימי היא מבצעת סריקה מכניסה את ה-value שלו למערך ואם אחד הבנים שלו או שניהם הם צמתים פנימיים בעץ הפונקציה תזומן גם עליהם עם קבלת index מתאים. לבסוף הפונקציה infoToArray תחזיר את המערך שקיבלה מפונקצית העזר subtreeValuesToArray. אצין שהמערך שיוחזר יהיה ממין כי עברנו על צמתי העץ בסריקת in-order.

**סיבוכיות זמן הריצה-**

במקרה הגרוע(העץ לא ריק) נעבור על כל הצמתים שלו באמצעות סריקת in-order ולכן הסיבוכיות תהיה  $O(n)$  כאשר n הוא מספר הצמתים בעץ.

**קריאה למתודות עזר-**

קראנו למתודה subtreeValuesToArray כפי שציינו.

## ○ size() -

**מה היא עושה-**

מחזירה את מספר האיברים שיש בעץ.

**כיצד היא פועלת-**

מחזירה את השדה size.

**סיבוכיות זמן הריצה-**

$O(1)$  שכן מתבצע מספר קבוע של פעולות.

## - select(int i)

**מה היא עושה-**

מחזירה את ערך ה value של האיבר עם המפתח ה-i בגודלו בעץ. במקרה שיש פחות מ-i צמתים בעץ יוחזר null.

**כיצד היא פועלת-**

בודקת אם המתודה size מחזירה ערך שקטן מ-i או אם i לא חיובי ממש ואם כן מחזירה null. אחרת יש קריאה לפונקציית העזר fingerSearch(int i) שמחפשת את הצומת עם המפתח ה-i בגודלו בעץ. מתודה זו מתחילה מהצומת עם המפתח המינימלי שמתקבל מביצוע פעולת min (שפשוט מחזירה את שדה המינימום). המתודה משתמשת בפונקציה getSubtreeSize() שמחזירה מה גודל תת העץ. ואם גודל תת העץ קטן מ-i עולים לצומת בשדה parent. כעת כשמצאנו צומת שתת העץ שלה בגודל שגדול או שווה ל-i יש קריאה לפונקציה select שמחפשת את הצומת בגודל ה-i באותו תת עץ. הפונקציה select פועלת באופן רקורסיבי. היא בודקת אם יש מספיק צמתים בתת העץ השמאלי ואם כן מזמנת אותה על תת העץ הימני אחרת היא מזמנת אותה על תת העץ הימני עם ערך i ששווה ל-i פחות 1 פחות מספר האיברים שיש בתת העץ השמאלי. כמובן שבכל ריצה אם מספר האיברים בתת העץ השמאלי שווה ל-i אז מוחזר אותו צומת. לבסוף, הפונקציה מחזירה את ה-value של אותו צומת. נציין שיש שתי פונקציות ששמן select אחת היא "פונקצית האם" שמקבלת איבר מטיפוס int והשנייה היא פונקצית עזר שעליה הסברתי שמקבלת גם צומת.

**סיבוכיות זמן הריצה-**

$O(\log(n))$  כי גם פונקציית העזר select וגם פונקציית העזר fingersearch עוברות על מספר צמתים ששווה במקרה הגרוע לגובה העץ.

**קריאה למתודות עזר-**

יש קריאה למתודות עזר כפי שציינו עבור כל מתודת עזר הסברתי מה היא עושה בסעיף "כיצד היא פועלת" שנמצא לעיל.

## - getRoot() ○

**מה היא עושה-**

מחזירה את שורש העץ.

**כיצד היא פועלת-**

מחזירה את השדה root .

**סיבוכיות זמן הריצה-**

$O(1)$  שכן מתבצע מספר קבוע של פעולות.

## מדידות

| מספר פעולות<br>האיזון<br>המקסימלי<br>delete לפעולת | מספר פעולות<br>האיזון<br>המקסימלי<br>insert לפעולת | מספר פעולות<br>האיזון<br>הממוצע<br>delete לפעולת | מספר פעולות<br>האיזון<br>הממוצע<br>insert לפעולת | מספר פעולות | מספר<br>סידורי |
|----------------------------------------------------|----------------------------------------------------|--------------------------------------------------|--------------------------------------------------|-------------|----------------|
| 13                                                 | 15                                                 | 3.450                                            | 3.428                                            | 10000       | 1              |
| 14                                                 | 16                                                 | 3.438                                            | 3.421                                            | 20000       | 2              |
| 15                                                 | 18                                                 | 3.435                                            | 3.412                                            | 30000       | 3              |
| 14                                                 | 19                                                 | 3.430                                            | 3.412                                            | 40000       | 4              |
| 14                                                 | 18                                                 | 3.440                                            | 3.414                                            | 50000       | 5              |
| 14                                                 | 18                                                 | 3.436                                            | 3.422                                            | 60000       | 6              |
| 15                                                 | 19                                                 | 3.422                                            | 3.423                                            | 70000       | 7              |
| 14                                                 | 19                                                 | 3.433                                            | 3.416                                            | 80000       | 8              |
| 14                                                 | 19                                                 | 3.424                                            | 3.422                                            | 90000       | 9              |
| 15                                                 | 19                                                 | 3.435                                            | 3.419                                            | 100000      | 10             |

### התוצאות שציפינו לקבל בטבלה

בהרצאה למדנו כי מספר פעולות האיזון שמתבצעות בעץ WAVL הוא  $O(1)$  amortized. משום כך ציפינו כי בסדרה בעלת פעולות גדולות וכאשר נבצע סדרה עולה של פעולות הכנסה ומחיקה כך גם מספר פעולות האיזון לא יעלה ויהיה די קבוע בממוצע ולכן גם לא תלוי בגודל הקלט וישאר קבוע.

## האם התוצאות שקיבלנו תואמות את הציפיות?

בטבלה ניתן לראות כי מספר פעולות האיזון הממוצע הן לפעולת הכנסה והן לפעולת מחיקה הוא קבוע ולא גדל כאשר יש מספר רב יותר של פעולות הכנסה/מחיקה או כאשר יש יותר צמתים בעץ. שכן עבור הכנסה ניתן לראות שמספר פעולות האיזון הממוצע סובב סביב 3.41 ועבור מחיקה ניתן לראות שמספר פעולות האיזון הממוצע סובב סביב 3.43. בנוסף, ניתן לראות שהמספר המקסימלי של פעולות איזון לא גדל כאשר הכנסנו יותר איברים משמע מספר פעולות האיזון לא גדל כאשר היו יותר צמתים בעץ הן להכנסה להן למחיקה. הדבר בא לידי ביטוי גם בממוצע פעולות האיזון שכן כאשר מספר הפעולות עולה (אף פי 10!) מספר פעולות האיזון לפעולה אינו משתנה בממוצע וקבוע, בהתאם ל-  $O(1)$ .

### משמעות המדידות שביצענו

המשמעות של המדידות אותן ביצענו היא שמספר פעולות האיזון בעץ WAVL, שאת הקוד שמתאר אותו כתבנו, אינן תלויות במספר האיברים אשר קיים בעץ ומספרן הוא קבוע בעבור כל הכנסה. חשיבות עובדה זו לא תסולא בפז עבור עץ WAVL עצום אשר ברצונינו להוסיף לו איבר או מנגד למחוק ממנו איבר כלשהוא.